Email: desigirl@umich.edu

For this homework, I implemented several mutation operators to modify the AST of the subject program and generate mutations to kill the tests based on the subject functions we were provided. Specifically, I implemented negation of comparison operations where < became >= and == became !=, swaps of binary operators where + became - and * became //, flips of boolean values where True became False, and targeted mutations of return statements to simulate deletion or changes of function results. For example, I added special-case mutations for select functions to guarantee that certain tests were killed. In function f04 I changed 'return g' to 'return g+1' and changed the function call in f05 from 'min(i,j)' to 'max(i,j).' These random mutations  applied to constants, boolean values, and other binary operations allowed tests to be killed selectively depending on which mutation was chosen.

The program decides which operator to apply by first choosing exactly one function to mutate, then selecting exactly one node (called a candidate node) in that function. Since I was having trouble killing some tests, I put nodes like f04 and f05 to be prioritized if present, ensuring that those corresponding tests were killed with the mutation. Other nodes were mutated randomly using the random.choice python function with operators matching the AST type. One of the biggest challenges I ran into was ensuring that each mutant killed exactly one test and not multiple. Initially, I implemented mutate.py so that mutations to helper functions or multiple nodes at once caused several tests to fail simultaneously because the mutations were not specific to one test which violated the "one mutation per test" criteria. Restricting mutations to a single node in one function solved this problem and required more thought about which nodes could be mutated without affecting other tests.

I think mutation analysis is a stronger indicator of code correctness than statement coverage because it forces developers to think about the logic of the program and how small faults affect behavior. While statement coverage can show that all lines are executed, it is unconsciously biased: a developer will most likely write tests that confirm what they think should be the intended behavior of the program rather than expose subtle faults because they only have to ensure most statements in the program are run. In contrast, mutation testing explicitly injects faulty logic and measures whether the test suite can detect them, which requires the tests to be thought of more carefully and will often make the developer realize if the current logic is fault-proof or needs to be revisited because a supposed fault was not caught when running the tests. Overall, mutation analysis is more adequate as a testing criteria than statement coverage, especially for developers testing code that they have participated in writing.